COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 09-04-2026 | **DATE SUBMITTED:** | 09-082026 |
| **NAME:** | Kherwin Emmanuel D. Milan | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

## Hands-on Activity 7.1 Sorting Algorithms Pt1

### 1. PROCEDURE OUTPUT

What procedures did you perform?

| | |
|---|---|
| Code + Console Screenshot | `srand(7);`<br><br>`int original[ARR_SIZE];`<br>`for(int i = 0; i < ARR_SIZE; i++){`<br>`original[i] = rand() % 1000;`<br>`}`<br><br>`Unsorted Array (100 elements)`<br>`61, 422, 215, 40, 521, 516, 152, 794, 727, 344,`<br>`818, 409, 42, 801, 42, 215, 63, 189, 812, 512,`<br>`580, 185, 455, 472, 374, 198, 201, 266, 140, 133,`<br>`453, 475, 54, 13, 945, 966, 234, 82, 741, 288,`<br>`727, 103, 153, 711, 435, 447, 719, 151, 227, 217,`<br>`985, 85, 890, 194, 699, 119, 859, 135, 876, 932,`<br>`58, 963, 530, 280, 176, 246, 564, 531, 957, 903,`<br>`235, 819, 596, 419, 499, 798, 25, 687, 294, 116,`<br>`597, 914, 139, 420, 550, 998, 941, 87, 840, 841,`<br>`174, 410, 729, 815, 934, 464, 780, 954, 855, 39` |
| Observations | It holds 100 randomly generated integers between 0 and 999. A fixed random seed was used so the same array could be reused and verified across every sorting technique |

Table 7-1. Array of Values for Sort Algorithm Testing

| | |
|---|---|
| Code + Console Screenshot | `template <typename T>`<br>`void bubbleSort(T arr[], size_t arrSize){`<br>`//Step 1: For i = 0 to N-1 repeat Step 2` |

```
    for(int i = 0; i < (int)arrSize; i++){
        //Step 2: For J = i + 1 to N - I repeat
        for(int j = i+1; j < (int)arrSize; j++){
            //Step 3: if A[J] > A[i]
            if(arr[j] > arr[i]){
                //Swap A[J] and A[i]
                std::swap(arr[j], arr[i]);
            }
            //[End of Inner for loop]
        }
        //[End of Outer for loop]
    }
    //Step 4: Exit
}
```

```
Bubble Sort Result
998, 985, 966, 963, 957, 954, 945, 941, 934, 932,
914, 903, 890, 876, 859, 855, 841, 840, 819, 818,
815, 812, 801, 798, 794, 780, 741, 729, 727, 727,
719, 711, 699, 687, 597, 596, 580, 564, 550, 531,
530, 521, 516, 512, 499, 475, 472, 464, 455, 453,
447, 435, 422, 420, 419, 410, 409, 374, 344, 294,
288, 280, 266, 246, 235, 234, 227, 217, 215, 215,
201, 198, 194, 189, 185, 176, 174, 153, 152, 151,
140, 139, 135, 133, 119, 116, 103, 87, 85, 82,
63, 61, 58, 54, 42, 42, 40, 39, 25, 13
```

| Observations | Bubble sort correctly moved the largest element to the front of the array on every outer pass, producing the array sorted from highest to lowest. |
|---|---|

Table 7-2. Bubble Sort Technique

| Code + Console Screenshot |
|---|

```
//Selection Sort (Array, Size of Array)
template <typename T>
void selectionSort(T arr[], const int N){
    int POS, temp, pass = 0;
    //Step 1: Repeat Steps 2 and 3 for K = 1 to N-1
    for(int i = 0; i < N; i++){
```

```
        //Step 2: Call routine smallest(A, K, N,
POS)
        POS = Routine_Smallest(arr, i, N);
        temp = arr[i];
        //Step 3: Swap A[K] with A[POS]
        arr[i] = arr[POS];
        arr[POS] = temp;
        //Count
        pass++;
    }
    //[End of loop]
    //Step 4: EXIT
}

//Routine smallest (Array, Current Position, Size
of Array)
template <typename T>
int Routine_Smallest(T A[], int K, const int
arrSize){
    int position, j;
    //Step 1: [initialize] set smallestElem = A[K]
    T smallestElem = A[K];
    //Step 2: [initialize] set POS = K
    position = K;
    //Step 3: for J = K+1 to N-1, repeat
    for(int J = K+1; J < arrSize; J++){
        if(A[J] < smallestElem){
            smallestElem = A[J];
            position = J;
        }
    }
    //Step 4: return POS
    return position;
}
```

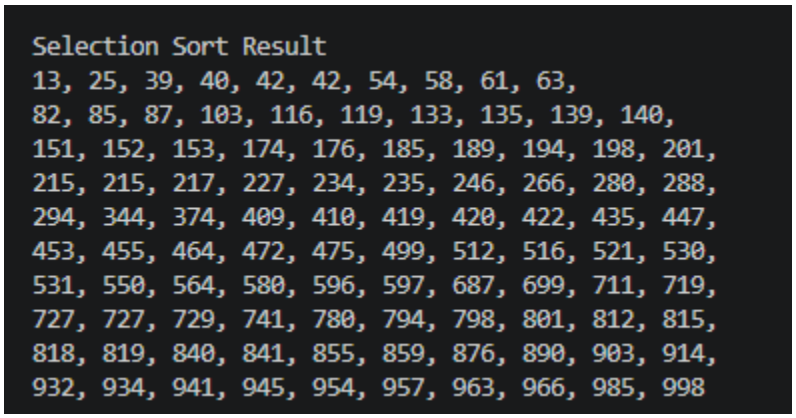
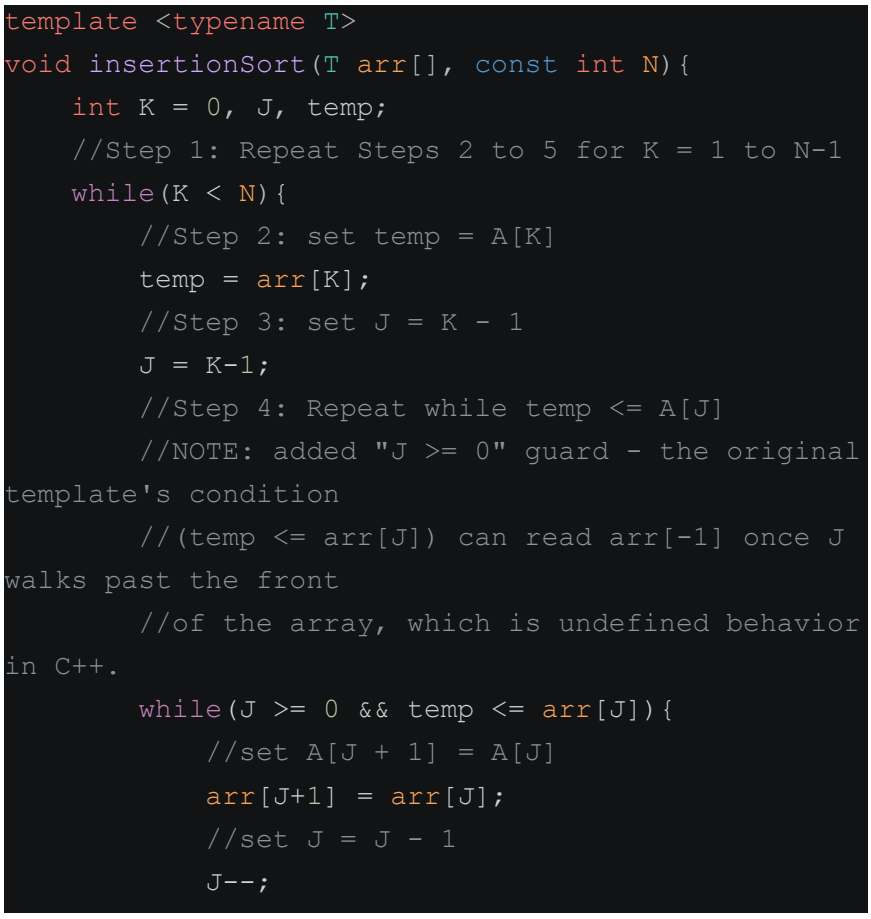
| | |
|---|---|
| |  |
| Observations | Selection sort produced the array in ascending order. It performed exactly N passes and, on each pass, scanned the remaining unsorted portion once to find the minimum, then did a single swap. |

Table 7-3. Selection Sort Algorithm

| | |
|---|---|
| Code + Console Screenshot |  |

```
            //[end of inner loop]
        }
        //Step 5: set A[J + 1] = temp
        arr[J+1] = temp;
        //[end of loop]
        K++;
    }
    //Step 6: exit
}
```

```
Insertion Sort Result
13, 25, 39, 40, 42, 42, 54, 58, 61, 63,
82, 85, 87, 103, 116, 119, 133, 135, 139, 140,
151, 152, 153, 174, 176, 185, 189, 194, 198, 201,
215, 215, 217, 227, 234, 235, 246, 266, 280, 288,
294, 344, 374, 409, 410, 419, 420, 422, 435, 447,
453, 455, 464, 472, 475, 499, 512, 516, 521, 530,
531, 550, 564, 580, 596, 597, 687, 699, 711, 719,
727, 727, 729, 741, 780, 794, 798, 801, 812, 815,
818, 819, 840, 841, 855, 859, 876, 890, 903, 914,
932, 934, 941, 945, 954, 957, 963, 966, 985, 998
```

| | |
|---|---|
| Observations | Insertion sort also produced the array in ascending order, matching the selection sort result exactly, which confirms both implementations are correct. |

Table 7-4. Insertion Sort Algorithm

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Pseudocode of Algorithm:

```
BEGIN VoteCount
  Declare votes[100]
  Declare tally[1..5] = 0
  Declare candidateNames[1..5]

  // Step 1: Generate votes
  FOR i = 0 TO 99
     votes[i] = random value from 1 to 5
  END FOR
```

```
  // Step 2: Sort the votes so equal values are grouped together
  CALL insertionSort(votes, 100)

  // Step 3: Count votes in a single linear pass over the sorted array
  FOR i = 0 TO 99
     tally[ votes[i] ] = tally[ votes[i] ] + 1
  END FOR

  // Step 4: Determine the winner
  winner = 1
  FOR c = 2 TO 5
     IF tally[c] > tally[winner] THEN
        winner = c
     END IF
  END FOR

  // Step 5: Output results
  PRINT tally[1..5] with candidateNames[1..5]
  PRINT "Winner: " + candidateNames[winner] + " with " + tally[winner] + " votes"
END VoteCount
```

Screenshot of Algorithm Code:

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int NUM_VOTES = 100;
const int NUM_CANDIDATES = 5;

const std::string candidateNames[NUM_CANDIDATES] = {
    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustin",
    "Deja Jayla Banaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

void printArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        std::cout << arr[i];
        if(i != size-1) std::cout << ", ";
```

```
        if((i+1) % 20 == 0) std::cout << "\n";
    }
    std::cout << std::endl;
}

int main(){
    srand(5);

    int votes[NUM_VOTES];
    for(int i = 0; i < NUM_VOTES; i++){
        votes[i] = (rand() % NUM_CANDIDATES) + 1;
    }

    std::cout << "Unsorted Vote Array (100 votes, candidate 1-5)\n";
    printArray(votes, NUM_VOTES);

    insertionSort(votes, NUM_VOTES);

    std::cout << "\nSorted Vote Array\n";
    printArray(votes, NUM_VOTES);

    int tally[NUM_CANDIDATES + 1] = {0}; // index 1..5 used

    for(int i = 0; i < NUM_VOTES; i++){
        tally[votes[i]]++;
    }

    std::cout << "\nVote Count Result\n";
    int winner = 1;
    for(int c = 1; c <= NUM_CANDIDATES; c++){
        std::cout << "Candidate " << c << " (" << candidateNames[c-1] <<
"): "
                  << tally[c] << " votes\n";
        if(tally[c] > tally[winner]) winner = c;
    }

    std::cout << "\nWinner: Candidate " << winner << " - "
              << candidateNames[winner-1] << " with " << tally[winner]
```

```
                << " votes.\n";


    return 0;
}
```

Output Testing:

```
Unsorted Vote Array (100 votes, candidate 1-5)
5, 4, 1, 5, 1, 1, 3, 5, 3, 1, 5, 2, 4, 2, 2, 3, 4, 3, 1, 2,
3, 3, 4, 2, 2, 5, 3, 4, 3, 2, 2, 5, 5, 1, 4, 1, 3, 3, 4, 5,
2, 2, 5, 1, 1, 5, 1, 1, 2, 3, 5, 1, 2, 5, 4, 5, 1, 5, 3, 2,
4, 2, 4, 2, 2, 1, 5, 1, 4, 2, 2, 1, 1, 4, 2, 3, 1, 1, 3, 1,
4, 1, 5, 4, 1, 1, 2, 5, 4, 4, 1, 2, 5, 3, 4, 4, 1, 5, 3, 1


Sorted Vote Array
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,
3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
4, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5


Vote Count Result
Candidate 1 (Bo Dalton Capistrano): 26 votes
Candidate 2 (Cornelius Raymon Agustin): 21 votes
Candidate 3 (Deja Jayla Banaga): 16 votes
Candidate 4 (Lalla Brielle Yabut): 18 votes
Candidate 5 (Franklin Relano Castro): 19 votes

Winner: Candidate 1 - Bo Dalton Capistrano with 26 votes.
```

Question: Was your developed vote counting algorithm effective? Why or why not?
Yes, the algorithm was effective. It correctly grouped and counted all 100 votes, the program's count matched exactly, and it correctly identified the candidate with the most votes.

Conclusion:
This activity helped me understand how bubble sort, selection sort, and insertion sort actually work by implementing them myself instead of just reading about them. I learned that even algorithms that sound similar can behave differently depending on how they are written.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.